Upgrade a customized database

Upgrading to a new version of Odoo can be challenging, especially if the database you work on contains custom modules. This page intent is to explain the technical process of upgrading a database with customized modules. Refer to **Upgrade documentation** for guidance on how to upgrade a database without customized modules.

We consider a custom module, any module that extends the standard code of Odoo and that was not built with the Studio app. Before upgrading such a module, or before requesting its upgrade, have a look at the **Service-level agreement (SLA)** to make sure who's responsible for it.

While working on what we refer to as the **custom upgrade** of your database, keep in mind the goals of an upgrade:

1. Stay supported
2. Get the latest features
3. Enjoy the performance improvement
4. Reduce the technical debt
5. Benefit from security improvements

With every new version of Odoo, changes are introduced. These changes can impact modules on which customization have been developed. This is the reason why upgrading a database that contains custom modules requires additional steps in order to upgrade the source code.

These are the steps to follow to upgrade customized databases:

1. **Stop the developments and challenge them**.
2. **Request an upgraded database**.
3. **Make your module installable on an empty database**.
4. **Make your module installable on the upgraded database**.
5. **Test extensively and do a rehearsal**.
6. **Upgrade the production database**.

Step 1: Stop the developments

Starting an upgrade requires commitment and development resources. If developments keep being made at the same time, those features will need to be re-upgraded and tested every

and compare them with the features introduced between your current version and the version you are targeting. Challenge the developments as much as possible and find functional workarounds. Removing redundancy between your developments and the standard version of Odoo will lead to an eased upgrade process and reduce technical debt.

> **Note**
>
> You can find information on the changes between versions in the **Release Notes**.

Step 2: Request an upgraded database

Once the developments have stopped for the custom modules and the implemented features have been challenged to remove redundancy and unnecessary code, the next step is to request an upgraded test database. To do so, follow the steps mentioned in **Obtaining an upgraded test database**, depending on the hosting type of your database.

The purpose of this stage is not to start working with the custom modules in the upgraded database, but to make sure the standard upgrade process works seamlessly, and the test database is delivered properly. If that's not the case, and the upgrade request fails, request the assistance of Odoo via the **support page** by selecting the option related to testing the upgrade.

Step 3: Empty database

Before working on an upgraded test database, we recommend to make the custom developments work on an empty database in the targeted version of your upgrade. This ensures that the customization is compatible with the new version of Odoo, allows to analyze how it behaves and interacts with the new features, and guarantees that they will not cause any issues when upgrading the database.

Making the custom modules work in an empty database also helps avoid changes and wrong configurations that might be present in the production database (like studio customization, customized website pages, email templates or translations). They are not intrinsically related to the custom modules and that can raise unwanted issues in this stage of the upgrade process.

To make custom modules work on an empty database we advise to follow these steps:

4. Make standard tests run successfully

Make custom modules installable

The first step is to make the custom modules installable in the new Odoo version. This means, starting by ensuring there are no tracebacks or warnings during their installation. For this, install the custom modules, one by one, in an empty database of the new Odoo version and fix the tracebacks and warnings that arise from that.

This process will help detect issues during the installation of the modules. For example:

- Invalid module dependencies.
- Syntax change: assets declaration, OWL updates, attrs.
- References to standard fields, models, views not existing anymore or renamed.
- Xpath that moved or were removed from views.
- Methods renamed or removed.
- ...

Test and fixes

Once there are no more tracebacks when installing the modules, the next step is to test them. Even if the custom modules are installable on an empty database, this does not guarantee there are no errors during their execution. Because of this, we encourage to test thoroughly all the customization to make sure everything is working as expected.

This process will help detect further issues that are not identified during the module installation and can only be detected in runtime. For example, deprecated calls to standard python or OWL functions, non-existing references to standard fields, etc.

We recommend to test all the customization, especially the following elements:

- Views
- Email templates
- Reports
- Server actions and automated actions
- Changes in the standard workflows
- Computed fields

We also encourage to write automated tests to save time during the testing iterations, increase the test coverage, and ensure that the changes and fixes introduced do not break the existing

Clean the code

At this stage of the upgrade process, we also suggest to clean the code as much as possible. This includes:

- Remove redundant and unnecessary code.
- Remove features that are now part of Odoo standard, as described in **Step 1: Stop the developments**.
- Clean commented code if it is not needed anymore.
- Refactor the code (functions, fields, views, reports, etc.) if needed.

Standard tests

Once the previous steps are completed, we advise to make sure all standard tests associated to the dependencies of the custom module pass. Standard tests ensure the validation of the code logic and prevent data corruption. They will help you identify bugs or unwanted behavior before you work on your database.

In case there are standard test failing, we suggest to analyze the reason for their failure:

- The customization changes the standard workflow: Adapt the standard test to your workflow.
- The customization did not take into account a special flow: Adapt your customization to ensure it works for all the standard workflows.

Step 4: Upgraded database

Once the custom modules are installable and working properly in an empty database, it is time to make them work on an **upgraded database**.

To make sure the custom code is working flawlessly in the new version, follow these steps:

- **Migrate the data**
- **Test the custom modules**

Migrate the data

During the upgrade of the custom modules, you might have to use **upgrade scripts** to reflect changes from the source code to their corresponding data. Together with the upgrade scripts, you can also make use of the **Upgrade utils** and its helper functions.

`rename_xmlid()`.

- Data from standard models removed from the source code of the newer Odoo version and from the database during the standard upgrade process might need to be recovered from the old model table if it is still present.

> **Example**
>
> Custom fields for model `sale.subscription` are not automatically migrated from Odoo 15 to Odoo 16 (when the model was merged into `sale.order`). In this case, a SQL query can be executed on an upgrade script to move the data from one table to the other. Take into account that all columns/fields must already exist, so consider doing this in a `post-` script (See **Phases of upgrade scripts**).
>
> > **Spoiler**

Upgrade scripts can also be used to:

- Ease the processing time of an upgrade. For example, to store the value of computed stored fields on models with an excessive number of records by using SQL queries.
- Recompute fields in case the computation of their value has changed. See also `recompute_fields()`.
- Uninstall unwanted custom modules. See also `remove_module()`.
- Correct faulty data or wrong configurations.

Running and testing upgrade scripts

Odoo Online | Odoo.sh | On-premise

As the instalation of custom modules containing Python files is not allowed on Odoo Online databases, it is not possible to run upgrade scripts on this platform.

Test the custom modules

To make sure the custom modules work properly with your data in the upgraded database, they need to be tested as well. This helps ensure both the standard and the custom data stored in the database are consistent and that nothing was lost during the upgrade process.

view needs to be activated again (or removed if not useful anymore). To achieve this, we recommend the use of upgrade scripts.

- **Module data** not updated: Custom records that have the `noupdate` flag are not updated when upgrading the module in the new database. For the custom data that needs to be updated due to changes in the new version, we recommend to use upgrade scripts to do so. See also: **`update_record_from_xml()`** .

Step 5: Testing and rehearsal

When the custom modules are working properly in the upgraded database, it is crucial to do another round of testing to assess the database usability and detect any issues that might have gone unnoticed in previous tests. For further information about testing the upgraded database, check **Testing the new version of the database**.

As mentioned in **Upgrading the production database**, both standard upgrade scripts and your database are constantly evolving. Therefore it is highly recommended to frequently request new upgraded test databases and ensure that the upgrade process is still successful.

In addition to that, make a full rehearsal of the upgrade process the day before upgrading the production database to avoid undesired behavior during the upgrade and to detect any issue that might have occurred with the migrated data.

Step 6: Production upgrade

Once you are confident about upgrading your production database, follow the process described on **Upgrading the production database**, depending on the hosting type of your database.

odoo docs